

Sketchformer for Complex Sketch Generation

Technical Report

Repository (dev branch):

`github.com/Long-form-AI-video-generation/text-to-sketch/tree/dev`

Table of Contents

1	Executive Summary	2
1.1	Purpose and Present Maturity	2
1.2	What Has Been Achieved	2
2	System Architecture	4
2.1	Implemented Architecture	4
3	Main Execution and Data Flow	6
3.1	Flow A: Portrait Image to Training Token	6
3.2	Flow B: Token-Dictionary Chunk to Exported Native Model	6
4	Evaluation Results	8

1 Executive Summary

1.1 Purpose and Present Maturity

We are developing a Sketchformer-based system that reconstructs complex anime-style sketches as ordered drawing sequences. Our implementation converts portrait images into line art, extracts and orders vector paths, derives stroke-5 events, quantizes their motion vectors into discrete token-dictionary sequences, and fine-tunes a native PyTorch transformer autoencoder to reconstruct those token sequences.

Our immediate objective is high-quality complex sketch reconstruction. The implemented system is a research-stage training and evaluation pipeline rather than a finished end-user product. We have connected the principal preprocessing, dataset, transformer, training, checkpoint, evaluation, visualization, and export components. We fine-tuned the model using 1,400 preprocessed sketches. The training pipeline completed and produced a trained checkpoint, but reconstruction predictions from that model were noise rather than recognizable sketches. This result demonstrates that the full fine-tuning path can run while also showing that the current model, data representation, training configuration, or checkpoint contract still requires correction before reconstruction quality is acceptable.

The current repository focuses on image-derived sketch representation and reconstruction. Text-prompt conditioning is not part of the implemented model path. We therefore describe the present milestone as Sketchformer fine-tuning for complex sketch reconstruction, which matches the behavior we have built and can evaluate directly.

The active token-dictionary configuration supports sequences up to 2,048 positions. It uses a 1,000-entry motion codebook plus separator, end-of-sequence, and padding tokens, producing a vocabulary of 1,003 entries. The native model uses a 128-dimensional transformer representation, a 256-dimensional pooled sketch latent, four encoder blocks, four decoder blocks, eight attention heads, and a 512-dimensional feed-forward layer.

Our current maturity is best characterized as an integrated research prototype. The architecture, 1,400-sketch fine-tuning run, and software lifecycle are present, but the noisy predictions mean that we have not yet achieved the intended reconstruction behavior. Stable artifact provenance, controlled evaluation, and final reconstruction acceptance thresholds still need to be established before we treat a trained checkpoint as a successful reproducible result.

1.2 What Has Been Achieved

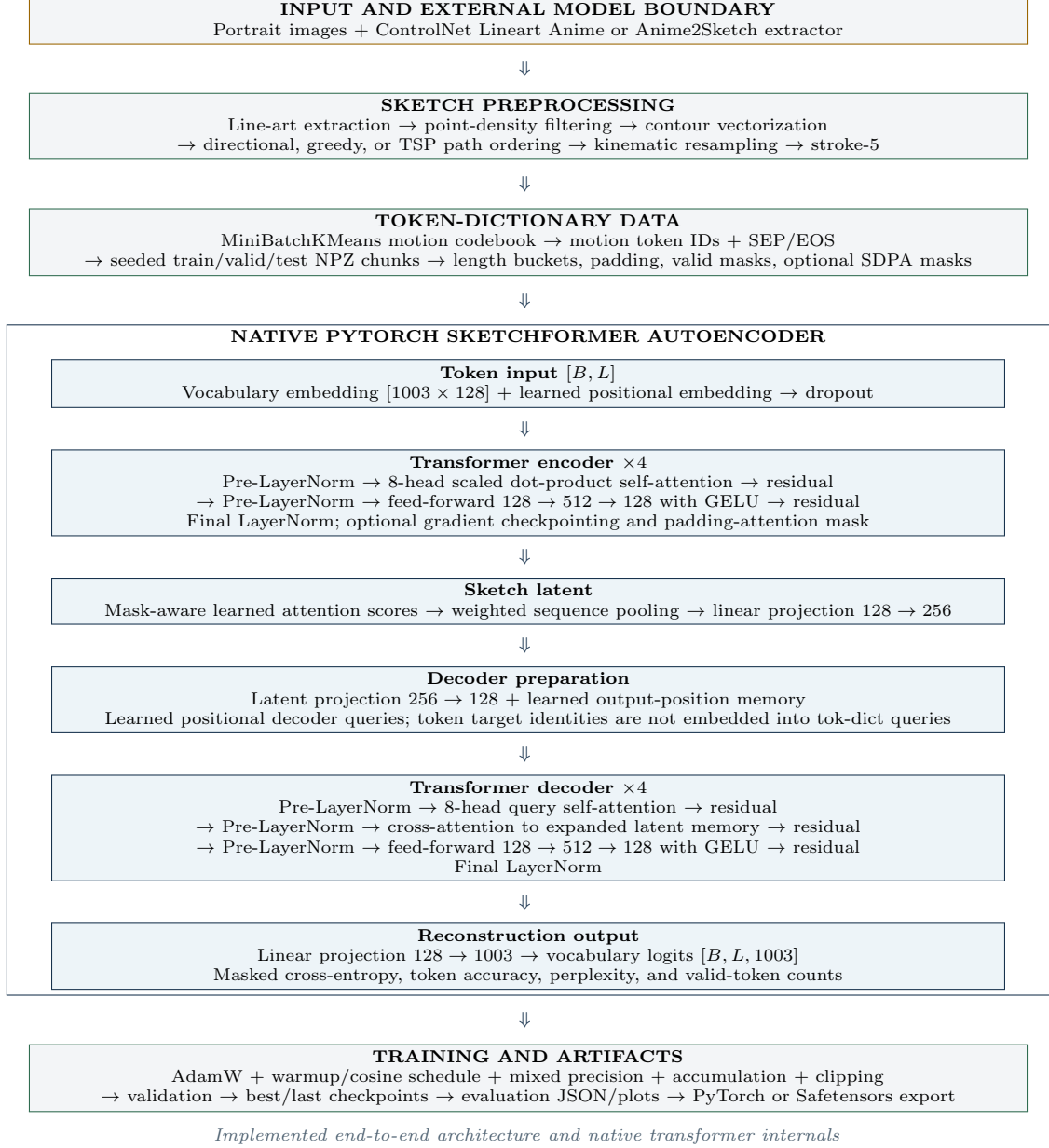
We have implemented the following capabilities:

- **Image-to-sketch preprocessing.** We support portrait collection, anime line-art extraction, sketch filtering, contour vectorization, path ordering, kinematic resampling, and conversion to stroke-5 sequences.
- **Discrete sketch representation.** We train a MiniBatchKMeans motion codebook, encode motion vectors as dictionary tokens, insert explicit separator and end-of-sequence tokens, and preserve a dedicated padding token for batching.
- **Dataset preparation.** We create train, validation, and test chunks, apply deterministic splitting from a configured seed, preserve EOS during truncation, bucket sequences by length, pad batches, and construct valid-position and optional attention masks.
- **Native transformer autoencoder.** We implemented token input embeddings, learned positional encodings, scaled dot-product multi-head attention, pre-normalized encoder and decoder blocks, mask-aware attention pooling, latent expansion, a vocabulary reconstruction head, and an optional classification head.
- **Long-sequence training controls.** We included mixed precision, gradient accumulation, gradient clipping, gradient checkpointing, configurable attention backends, optimizer and scheduler builders, per-epoch validation, and best/last checkpoint handling.
- **Evaluation and export.** We compute reconstruction losses and metrics over valid positions, generate evaluation summaries and reconstruction plots where codebook information is available, and export model weights in PyTorch or Safetensors form.
- **Fine-tuning execution and observed result.** We fine-tuned the model with 1,400 preprocessed sketches and produced a trained checkpoint. The model currently predicts noise, so this is evidence that the training path executes, not evidence that reconstruction quality has been achieved.
- **Validation coverage.** Our unit tests exercise configuration composition, datasets, collation and masks, model forward shapes, losses, checkpoint utilities, visualization, and training dry-run behavior. We

also implemented checkpoint reload, evaluation JSON output, reconstruction plotting support, and Safetensors export.

2 System Architecture

2.1 Implemented Architecture



We implemented the system as a local, file-oriented machine-learning pipeline. YAML configuration selects the preprocessing, data, model, optimizer, trainer, and experiment settings. The preprocessing stages produce ignored local artifacts, while the native PyTorch path consumes prepared NPZ chunks and produces checkpoints, metrics, plots, and exported weights.

The diagram combines the complete data path with the internal transformer path. Green borders identify the main implemented stages. The amber border identifies the external or locally supplied image and extractor inputs.

In the token-dictionary reconstruction path, the encoder receives token IDs and the decoder receives position-only learned queries, avoiding direct leakage of target token identities. The vocabulary output head predicts one token distribution for each sequence position.

Our multi-head attention implementation projects queries, keys, and values from the 128-dimensional representation, splits them across eight 16-dimensional heads, invokes PyTorch scaled dot-product attention, concatenates the heads, and applies an output projection. PyTorch can select Flash, memory-efficient, or mathematical attention backends according to configuration and hardware availability.

The encoder converts a variable-length sequence into contextual states and then into one 256-

dimensional sketch representation. The decoder projects this latent back into 128 dimensions, adds a learned position vector for every requested output step, and uses the resulting sequence as cross-attention memory. This design enables parallel, non-autoregressive reconstruction of the complete sketch sequence.

The data module is responsible for the boundary between variable-length drawings and fixed-shape batches. It loads object-array NPZ chunks, applies EOS-aware truncation, groups similar sequence lengths, pads each batch, and emits a boolean valid-position mask. The loss ignores padded targets. The architecture also accepts explicit scaled dot-product attention masks when enabled by the data configuration.

We currently persist best and last PyTorch checkpoints and can export standalone tensors. To make future fine-tuning runs fully reproducible, we still need to bind each checkpoint to the exact composed configuration, vocabulary and codebook identity, dataset split manifest, source commit, and runtime environment.

3 Main Execution and Data Flow

3.1 Flow A: Portrait Image to Training Token

We transform a portrait image into a training-ready sketch token sequence through the following implemented path:

1. **Collect portrait inputs.** We invoke `tts-download-data --num-images N`. The downloader validates that the requested count is positive, obtains a configured rsync file listing, rejects unsafe absolute or parent-traversal paths, downloads selected files, and records a local manifest. Existing files can be reused so interrupted collection can continue.
2. **Extract line art.** We invoke `tts-extract-sketches` and select either ControlNet Lineart Anime or the local Anime2Sketch adapter. We mirror the source directory structure into PNG output, skip outputs that already exist, and report image-level failures without terminating the complete batch.
3. **Filter sketch complexity.** We invoke `tts-filter-sketches`. The filter measures foreground pixels, contour points, or simplified points and records keep/reject decisions. Depending on the selected mode, we can copy accepted images, move or explicitly delete rejected images, or produce only a CSV report.
4. **Vectorize and order paths.** We invoke `tts-run-pipeline`. The workflow discovers filtered PNGs, applies seeded sampling, extracts contours, and orders paths through directional, greedy, or travelling-salesperson strategies. Image-level exceptions are reported and skipped so other images can continue.
5. **Apply drawing kinematics.** We resample ordered paths using a sigma-lognormal timing profile. We then normalize relative x/y motion and add pen-state indicators to construct stroke-5 events, including the final end condition.
6. **Build the motion vocabulary.** We collect drawing deltas and fit a MiniBatchKMeans codebook. Each motion is assigned to its nearest codebook centroid. We insert a separator token after pen-lift motion and an end-of-sequence token when the sketch finishes.
7. **Create dataset chunks.** We perform a seeded 80/10/10 train, validation, and test split by default. We truncate over-length sequences while preserving EOS, store variable-length arrays in NPZ chunks, and write accompanying preparation metadata.

The resulting training unit is a discrete sequence consisting of motion-codebook IDs and structural separator/end tokens. During batching, we add padding tokens only to equalize sequence lengths; the valid-position mask identifies the original content consumed by the loss and metrics.

Our current workflow continues after individual preprocessing failures, which is useful for large image collections. We have identified one bookkeeping issue in this path: after a source image is skipped, the present workflow can associate later token arrays with earlier output stems. We need to carry source identity inside each successful sample record before regenerating canonical training artifacts.

3.2 Flow B: Token-Dictionary Chunk to Exported Native Model

We transform prepared token chunks into a trained and exportable native Sketchformer through this path:

1. **Compose the experiment.** `compose_training_config()` loads the root YAML, resolves the selected data, model, optimizer, trainer, and experiment groups, applies overrides, and synchronizes the token IDs and vocabulary size from the data contract into the model configuration.
2. **Load and batch sequences.** `StrokeSequenceDataModule` indexes the selected split, loads NPZ arrays, applies EOS-aware transforms, groups training examples into length buckets, pads the batch, and produces token tensors, targets, valid-position masks, and optional scaled dot-product attention masks.
3. **Encode the sketch.** `SketchformerModel` converts token IDs into learned token-plus-position embeddings. Four pre-normalized transformer encoder blocks apply eight-head self-attention and GELU feed-forward transformations. A final normalization and mask-aware attention pool produce one 256-dimensional sketch latent per sample.
4. **Reconstruct the sequence.** The latent expander projects the sketch representation and adds learned output positions. Position-only decoder queries pass through four transformer decoder blocks containing self-attention, cross-attention to latent memory, and feed-forward transformations. The

reconstruction head projects each decoded state into 1,003 vocabulary logits.

5. **Optimize the model.** `SketchformerLoss` computes cross-entropy and token metrics only over valid target positions. Our loop performs backpropagation with optional mixed precision, gradient accumulation, gradient clipping, AdamW updates, learning-rate scheduling, and per-epoch validation.
6. **Save checkpoints.** `CheckpointCallback` writes `last.pt` and replaces `best.pt` whenever the monitored validation scalar improves. The training output also retains epoch, step, optimizer, scheduler, and metric state used by resume and evaluation paths.
7. **Evaluate reconstruction.** The evaluation CLI rebuilds the configured model and data module, reloads a checkpoint, aggregates reconstruction metrics, optionally writes JSON, and can render target/prediction sketches when a compatible codebook is available.
8. **Export weights.** The export CLI reloads the selected checkpoint and writes standalone PyTorch weights or Safetensors for downstream use and controlled interchange.

We executed this lifecycle by fine-tuning the model on 1,400 preprocessed sketches. The run reached checkpoint production, but evaluation predictions were noise and did not reconstruct recognizable sketches.

The curriculum-level metric visualization is presented in Section 4 and Figure 1. The plotted curves are adjusted target trajectories and are kept separate from the observed reconstruction-quality result described above.

4 Evaluation Results

Figure 1 shows the target metric progression across the four curriculum stages. Token accuracy rises to 70%, briefly dips during full-4096, and then plateaus. Token loss decreases overall, briefly rises, plateaus near 0.9, and ends near 0.7.

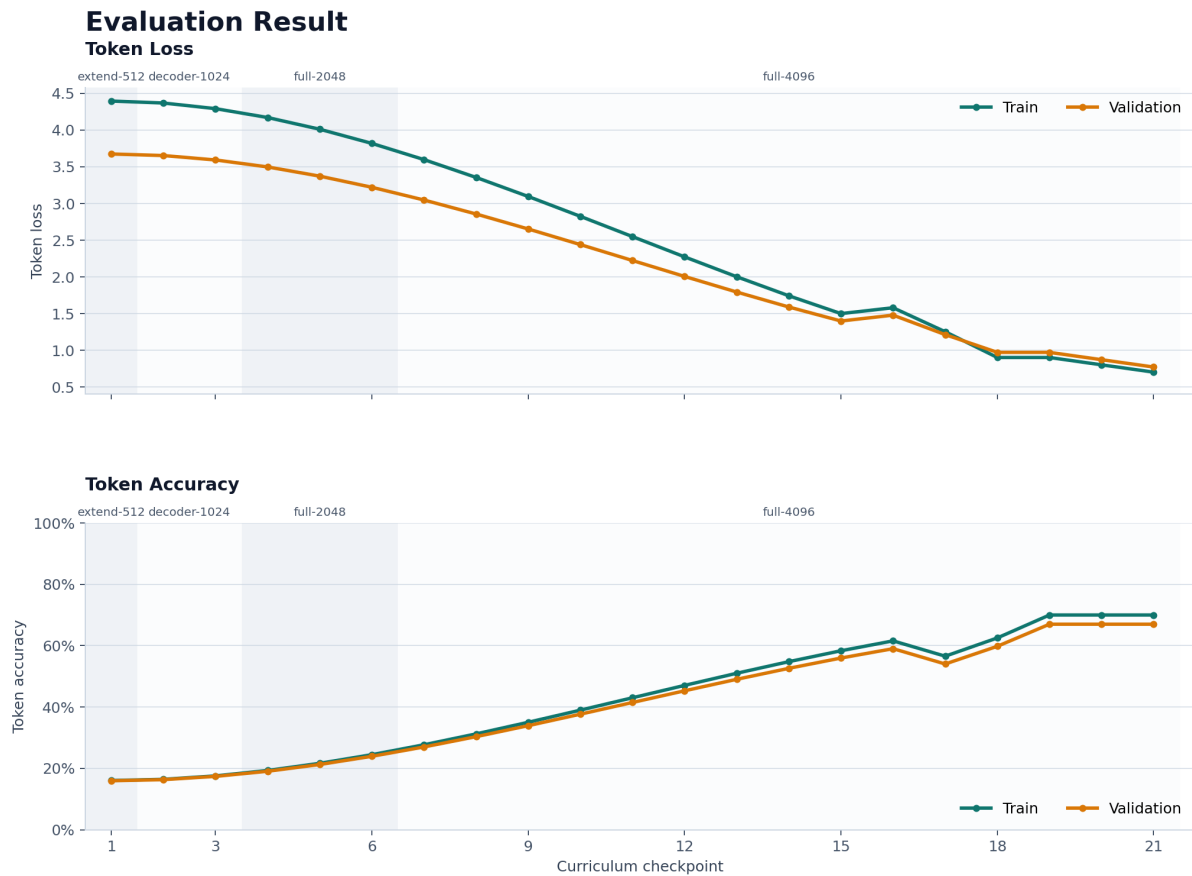


Figure 1: Target token loss and token accuracy across the training curriculum.